

Evaluation Pipeline Walkthrough

Overview

The Evaluation Pipeline has been fully implemented based on the modified requirements. The pipeline serves as a development tool (placed in `src/evaluation/`) to systematically assess the `DiseaseDetectionAgent` and automatically generate detailed visual, tabular, and JSON reports.

Changes Made

- **Separation of Concerns:** Moved the evaluation pipeline out of `agents/` and created `src/evaluation/`.
 - `[metrics.py](file:///d:/agricare/src/evaluation/metrics.py)` handles scikit-learn logic, fetching Accuracy, Precision, Recall, F1-Score, weakest/strongest classes, and top confused disease pairs.
 - `[report_generator.py](file:///d:/agricare/src/evaluation/report_generator.py)` generates structured outputs like `metrics.json`, `classification_report.txt`, the `confusion_matrix.png` plot, and the requested `model_health_report.md`.
 - `[evaluate.py](file:///d:/agricare/src/evaluation/evaluate.py)` iterates over any test dataset (with class-named folders), passes images to `DiseaseDetectionAgent.predict()`, records misclassifications into a CSV, and delegates report generation to the other modules.
- **Prefix Support:** The `evaluate.py` script accepts `--prefix` (e.g., `plantvillage` or `realworld`) and `--dataset_dir` to generate segregated reports.
- **Misclassified Images CSV:** Outputs `reports/[prefix]_misclassified_images.csv` with the format `image_path,predicted_class,actual_class,confidence` ready for the Feedback Agent.
- **Model Health Report:** Generates `reports/[prefix]_model_health_report.md` automatically aggregating accuracy, top confusions, strong/weak classes, and explicit data augmentation recommendations.
- **Testing:** Added `[test_evaluation_pipeline.py]` (`file:///d:/agricare/tests/test_evaluation_pipeline.py`) which simulates a dummy

dataset, runs the full evaluation script, and asserts that all 5 report artifacts are properly generated in the specified output directory.

Validation Results

- Ran `venv\Scripts\python -m unittest tests/test_evaluation_pipeline.py` .
- Ran 2 tests in 7.217s — All tests passed.
- The system correctly generates:
 - `dummy_classification_report.txt`
 - `dummy_confusion_matrix.png`
 - `dummy_metrics.json`
 - `dummy_misclassified_images.csv`
 - `dummy_model_health_report.md`

You can run the pipeline on your actual datasets using:

```
# For PlantVillage test set
venv\Scripts\python src/evaluation/evaluate.py --dataset_dir data/test --prefix plantvillage

# For Real-World test set
venv\Scripts\python src/evaluation/evaluate.py --dataset_dir data/real_world_test --pref
```